

Amanorsac Studio — Release Standard

Version 1.1 · October 2026 · Applies to every Amanorsac app, plug-in and plug-in bundle, paid or free.

This is how a build becomes a release on amanorsac.studio. It sits beside the License Integration Standard (`license-integration-for-developers.md`) and the Hub notes (`hub-integration.md`). Those say what a build must do; this says how a finished build reaches the website, the Hub and the buyers, and what it must bring with it.

The words **MUST** and **MUST NOT** mark requirements. Everything else is explanation.

The one rule. An app is not released until its repo has run the release steps in §4 and the website chat has received the release package. Nothing is pushed to R2 by hand, no page is written from memory, and no version number is typed into `catalog.json` by anyone but the website's receiver.

How to put this in an app repo. Save this file as `docs/release-standard.md` in the repo and add one line to the repo's `CLAUDE.md` :

Releases follow docs/release-standard.md. When asked to release a version, do §4 of it, in order, and stop where it says stop.

That is the whole setup. It works the same in a Claude Code chat on the studio's computer and in a cloud chat on the repo; no plugin is needed. The website repo carries the receiver's side (§6) the same way.

1. How it works

1. The app is built and tested **on the studio's own computer**. That is where it is recorded: the app is launched, each main screen is captured, and a short recording of it in use is taken.
2. The repo is pushed to GitHub, and GitHub's build makes the **final installers** (the signed Windows installer, the notarized Mac disk images). Mobile apps get their screenshots from the repo's own store assets instead of a recording.
3. In the repo's Claude Code chat, "release " gathers the installers, the screenshots, the recording, the product video and the hand-off notes into one **release package** and uploads it to the studio's R2 bucket under `releases/<app>/<version>/`.

4. The person tells the website chat `receive <app> <version>`, and it picks the package up.

Amanorsac puts the installers under the

catalog's exact names and confirms they are in the bucket, converts the screenshots for the web, places the video on the app's page, updates the catalog version and the page's "What's new" from the notes, and pushes the result to the website's working branch.

5. **Nothing goes live on its own.** The website chat reports what changed and asks for two things: the go-ahead, and the price (or a confirmation that the price is unchanged). Only after both does it fast-forward `main`.

2. What a release package MUST contain

Everything lives under `releases/<app>/<version>/` in the `amanorsac-downloads` bucket.

`<app>` is the catalog id (`chordlight88`, `nebulatide2`, ...) and `<version>` is the plain version (`2.0.4`).

Path	What	Rule
<code>handoff.md</code>	the hand-off notes, §3	MUST be present and complete
<code>installers/</code>	the final installers from GitHub, one per platform	MUST be the GitHub build's own files, not a local build; names as they came off the build
<code>screenshots/NN-<what>.png</code>	each main screen, numbered in the order a new user meets them	MUST be the real app, at its real size, no placeholder text, no developer paths on screen
<code>recording.mp4</code>	30–90 seconds of the app in use, no narration, system audio only if the app makes sound	desktop apps MUST; mobile apps MAY
<code>brag/brag.mp4</code> , <code>brag/brag.jpg</code> , <code>brag/share-copy.txt</code>	the product video, its poster, the caption	MUST, made by <code>/brag</code> from the repo (§5)

A package missing a MUST is not picked up; the receiver says what is missing and waits.

3. The hand-off notes (`handoff.md`)

Written by the app's chat, read by the website's. Plain headings, no prose padding. These sections, in this order:

```
# <App name> <version>
Amanorsac Studio · Release Standard v1.1
```

Release

- app: <catalog id>
- version: <x.y.z>
- date: <YYYY-MM-DD>
- platforms: windows, mac-arm64, mac-x64 (or: windows, mac / android / ios)
- licensed: yes | no (does this build ask for a key, per the licence standard)
- free: yes | no
- price: unchanged | <ask> (the price is the studio's call; say "unchanged" or "ask")

What's new

- one line per change a buyer would notice, in their words, most important first
- (an update email goes out only if the section below says so)

Email owners

yes | no
<one sentence for the email, if yes>

Installers

platform	file	bytes	sha256	
windows	ChordLight-88-2.0.4-Windows.exe	136092949	...	
mac-arm64	ChordLight-88-2.0.4-macOS-arm64.dmg	...	...	

Screenshots

file	shows	
01-home.png	Home: the drop zone and the library	

Website

- anything the page must say differently now (a renamed feature, a removed one, a new requirement)
- anything that MUST NOT appear in public yet (a name not cleared, a feature held back)

Known issues

- what a buyer might hit, and what to tell them

The receiver reads `version`, `platforms`, `licensed`, `free` and the Installers table mechanically. Everything else it reads as a person would and turns into the page.

4. The release steps (in every app repo)

Told "release 2.0.4" (or `/release 2.0.4` where the studio's mod is installed), the repo's chat MUST do these, in order, and MUST stop at the first one that fails rather than carry on:

1. **Check the build.** Confirm the version in the code matches, the GitHub build for that tag succeeded, and the installers exist. Stop and say so if not.
2. **Fetch the installers** from the GitHub release (or the workflow's artifacts) into `release/<version>/installers/`. Record size and sha256 for each.
3. **Record the app** (desktop apps, on the studio's computer): launch the built app, capture every main screen to `screenshots/`, and record 30–90 seconds of use to `recording.mp4`. Mobile apps: copy the store screenshots from the repo instead.
4. **Make the product video:** run `/brag` on the repo with the screenshots and recording as its material, into `brag/`. The brag video MUST use the real screens, never a mock-up.
5. **Write `handoff.md`** (§3) from the commit log since the last release and the repo's own README, then show it to the person for a read before it goes anywhere. A cloud chat that cannot launch the app says so here and asks for the screenshots and recording to be dropped into `release/<version>/` from the studio's computer, then continues.
6. **Upload** the package to `r2:amanorsac-downloads/releases/<app>/<version>/` with `rclone`, then list the folder back and compare bytes to what was sent. Stop if anything differs.
7. **Tell the website chat:** the person pastes one line into the website repo's chat, `receive <app> <version>`. That line is the hand-off; the website chat then does §6.

It MUST NOT upload anything under `<app>/` (the folder the catalog serves from). That folder is the receiver's alone.

5. Product video and screenshots

`/brag` (github.com/latent-spaces/brag) is the studio's product-video tool. Each release package carries a brag video so the page, the announcement and the socials have something to show on day one.

- The video MUST be built from the release's own screenshots and recording, so what it shows is the build being shipped.
 - 18–25 seconds, landscape 1920×1080. A vertical cut MAY be added as `brag/brag-vertical.mp4`.
 - Every line on screen MUST be the app's own wording (its README, its page, its UI). No invented numbers, claims or testimonials.
 - Nothing in the recording or screenshots may show a developer path, a licence key, an email address or test data a buyer would recognise as fake.
 - Music is the mod's own or the studio's; nothing licensed from a library without the licence saved beside it.
-

6. What the website's receiver does with it

Amanorsac Studio · Release Standard v1.1

On the hand-off message, the website chat MUST:

1. Read `handoff.md` . Refuse the package, naming the gap, if §2 is not met.
2. Copy each installer to its catalog name under `<app>/` and list the bucket back. Only after the listing shows the files with the right byte counts does `catalog.json` change. The previous version's keys stay as `alt_keys` .
3. Convert screenshots to webp at 1600 and 800 wide into `images/apps/<app>/` , place the brag video in `video/` , and put the video on the page the way the Easy Stems page does: muted, looped, no controls, playing only while on screen.
4. Set the catalog `version` , add a "New in " section to the page from What's new, add a `news` entry, and add a `notice` block only if `Email owners` says yes.
5. Push to the working branch and run the site checks.
6. Report to the person: what changed, the price question, and the go-ahead question. Fast-forward `main` only when both are answered.

It MUST NOT set a price, change a status to available, or push `main` without those answers.

7. Checklist, one line each

- ☐ built and tested on the studio's computer
 - ☐ pushed; GitHub build green; installers downloaded from that build
 - ☐ screenshots of every main screen, real data, nothing private on screen
 - ☐ recording made (desktop) or store screenshots copied (mobile)
 - ☐ `/brag` run from the repo; video, poster and caption in `brag/`
 - ☐ `handoff.md` written and read through
 - ☐ package in `releases/<app>/<version>/` , listed back, bytes match
 - ☐ website chat told: `receive <app> <version>`
 - ☐ website chat's report read; price given; go-ahead given
 - ☐ launch compliance (§8) checked for anything the release adds: a new email, a new third-party script, a new way to pay, a new upload path
-

8. Launch compliance

Amanorsac Studio · Release Standard v1.1

Six things that get small studios sued or fined, and the studio's answer to each. They are checked once for the website and again whenever a release touches the thing in question. The website chat MUST NOT push `main` with any of these broken.

#	The risk	The rule	Where it lives
1	Fonts loaded from a third-party CDN send every visitor's IP address to that company without consent (the German Google Fonts rulings).	Fonts are self-hosted under <code>/fonts/</code> and declared in the site's own CSS. No page links to <code>fonts.googleapis.com</code> or any font CDN.	<code>assets/site.css</code> , <code>assets/fonts.css</code>
2	Session replay and keystroke recording (Hotjar, FullStory, LogRocket and the like) record what people type before they press send.	None is used. The website's own first-party traffic record is what the privacy policy §2.4 describes and nothing more; a new third-party script of any kind MUST be named in privacy policy §5 before it ships.	<code>privacy.html</code> §2.4, §5
3	Marketing email without consent, without a stop link, or without a postal address breaks CAN-SPAM in the US and GDPR/PECR in Europe.	Studio news goes only to <code>public.subscribers</code> rows with <code>consented = true</code> and no <code>unsubscribed_at</code> ; an account alone is not consent. Every email the Worker sends carries a one-click stop link where it is not a transactional email, a <code>List-Unsubscribe</code> header, and the postal address line (<code>POSTAL</code> in <code>worker.js</code>). The sign-up box for news is unticked by default and never required.	<code>worker.js</code> , <code>supabase-announcements.sql</code> , <code>client.html</code>
4	Subscriptions that are easy to start and hard to stop (the FTC's click-to-cancel rule and the state auto-renewal laws).	The studio sells lifetime licences and no subscriptions. If one is ever added, cancelling MUST be one click from My Apps, as easy as signing up, with the renewal date and price shown before every charge.	<code>legal.html</code> §2.5

Amanorsac Studio - Release Standard v1.1

#	The risk	The rule	Where it lives
5	Hosting other people's uploads without a copyright agent loses the DMCA safe harbour for the store's creator packs.	Terms §2.7 names the designated agent, the address and what a notice must contain; the same agent is registered with the US Copyright Office DMCA directory (renewed every three years). A complete notice is acted on promptly and the uploader told; repeat infringers lose their store account.	<code>legal.html</code> §2.7
6	Children's data (COPPA): an account made by someone under 13 without parental consent.	Sign-up has a required "I am 13 or older and agree to the terms" box, checked in the page's own code, not left to the browser. The privacy policy says accounts are not made for anyone under 13 and how to have a child's data removed.	<code>client.html</code> , <code>privacy.html</code> §9

Two more that are not in that list but belong with it:

- **The privacy policy names every company that touches data.** Adding a payment provider, an email service, a host or an analytics tool means adding a line to `privacy.html` §5 in the same commit.
- **Prices and claims on a page are the product's own.** A release that renames or removes a feature changes the page in the same commit, per §6 step 4, so the page never promises what the build no longer does.